

SEL0615 – Processamento Digital de Sinais

Exercício 5

Bárbara Fernandes Madera
Nº USP: 11915032

1 Introdução

O presente relatório tem como objetivo resolver o Exercício 5 da disciplina SEL0615 – Processamento Digital de Sinais. O exercício aborda dois temas principais: o processamento de imagens RGB no domínio da frequência e a compressão de sinais de áudio utilizando a Transformada de Fourier.

Neste sentido, a primeira parte consiste em analisar os canais de cor de uma imagem e aplicadas máscaras no domínio da frequência. Já a segunda parte realiza a compressão de um sinal de áudio por meio da retenção de uma porcentagem dos coeficientes da FFT.

2 1) Processamento de Imagem RGB

Nesta etapa, foi realizada a análise de uma imagem colorida no formato RGB, com o objetivo de explorar suas componentes de cor e aplicar técnicas de filtragem no domínio da frequência. Inicialmente, a imagem foi decomposta em seus três canais (vermelho, verde e azul), permitindo observar a contribuição individual de cada componente. Em seguida, foram construídas máscaras circulares no domínio da frequência, as quais foram aplicadas separadamente a cada canal por meio da Transformada Rápida de Fourier (FFT). Por fim, os canais filtrados foram re combinados com uso de filtros distintos, resultando em novas imagens que evidenciam o impacto da filtragem espectral sobre os níveis de detalhe e suavização da imagem original.

2.1 (a) Separação dos canais de cor

A imagem *imagem_cao.jpg* foi carregada no MATLAB e decomposta em seus três canais de cor: vermelho (R), verde (G) e azul (B). Cada canal representa a intensidade de uma componente de cor da imagem.

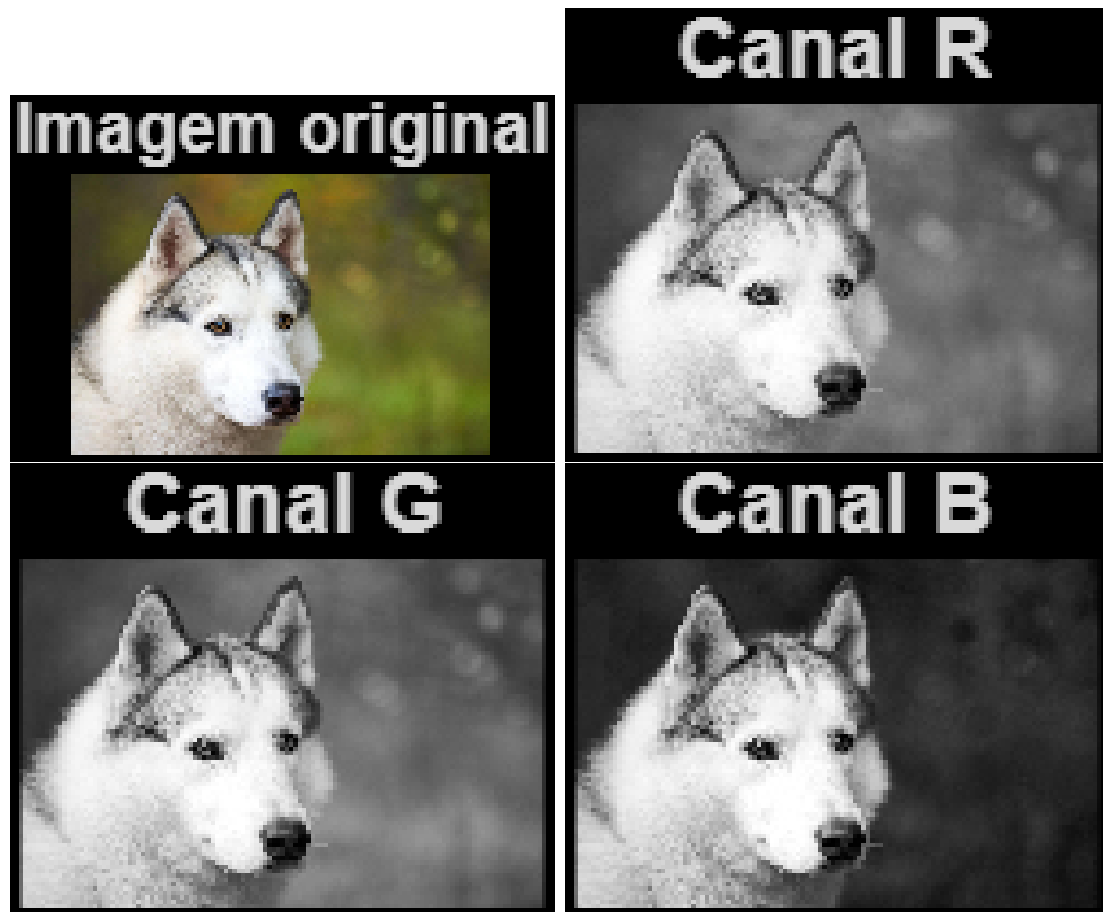


Figura 1: Imagem original e separação dos canais R, G e B.

2.2 (b) Criação das máscaras

Foram também criadas duas máscaras circulares no domínio da frequência, com raios de 25 e 45 pixels. Essas máscaras permitem selecionar regiões específicas do espectro da imagem.

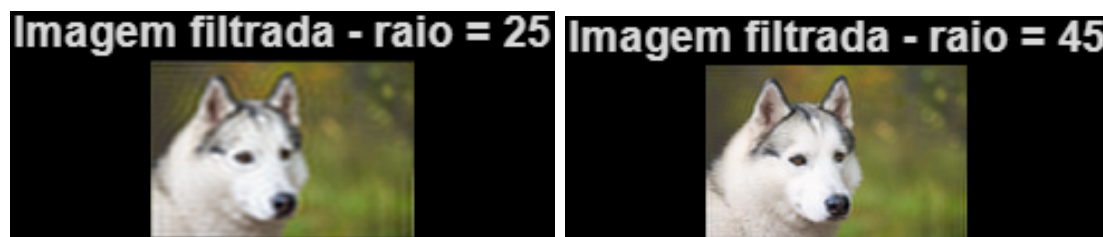


Figura 2: Máscaras circulares utilizadas para filtragem.

2.3 (c) Aplicação das máscaras

As máscaras foram aplicadas a cada canal no domínio da frequência por meio da Transformada Rápida de Fourier (FFT). Após a filtragem, os canais foram recombinados para formar as imagens RGB resultantes.

Os valores de raio utilizados (25 e 45 pixels) foram escolhidos de forma a permitir a comparação entre diferentes níveis de filtragem no domínio da frequência. O raio menor

atua como um filtro mais restritivo, preservando apenas componentes de baixa frequência e resultando em maior suavização da imagem. O raio maior, por outro lado, permite a passagem de uma faixa mais ampla de frequências, preservando mais detalhes da imagem original. Dessa forma, a escolha desses valores possibilita perceber e analisar de forma qualitativa o impacto do tamanho da máscara na reconstrução da imagem.



Figura 3: Imagem original e imagens filtradas com diferentes máscaras.

Observa-se que o aumento do raio da máscara permite a preservação de mais componentes de frequência, resultando em maior riqueza de detalhes na imagem reconstruída.

3 2) Compressão de Áudio

Nesta etapa subsequente, foi realizada a análise e compressão de um sinal de áudio por meio da Transformada Rápida de Fourier (FFT), permitindo sua representação no domínio da frequência. A partir dessa representação, foram aplicadas técnicas de compressão baseadas na redução do número de coeficientes utilizados na reconstrução do sinal. Especificamente, foram considerados cenários mantendo 5% e 1% dos coeficientes mais relevantes, possibilitando avaliar o impacto da compressão na qualidade do áudio. Com isso, os sinais reconstruídos foram analisados tanto no domínio do tempo quanto no da frequência, permitindo comparar visualmente e perceptivamente as perdas de informação decorrentes da compressão.

3.1 (a) Compressão com 5%

Foi realizada a compressão mantendo 5% dos coeficientes mais relevantes do sinal.

3.2 (b) Compressão com 1%

Foi realizada a compressão mantendo apenas 1% dos coeficientes, resultando em maior perda de informação.

3.3 Visualização no domínio do tempo

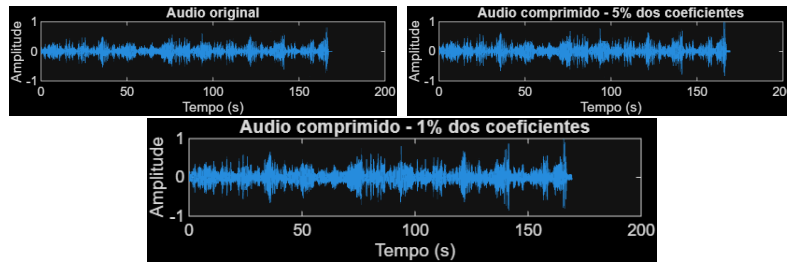


Figura 4: Sinal original e sinais comprimidos (5% e 1%).

3.4 Visualização no domínio da frequência

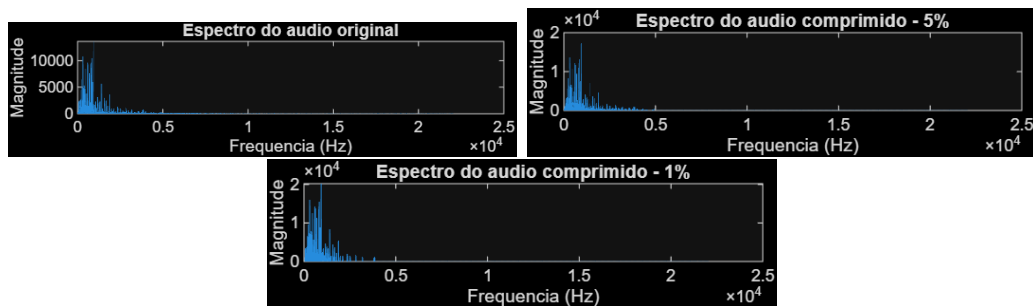


Figura 5: Espectros de frequência dos sinais de áudio.

3.5 (c) Análise perceptiva

Ao se comparar os sinais de áudio comprimidos com o original, observa-se que:

- Com 5% dos coeficientes, o áudio mantém boa qualidade, preservando suas principais características.
- Já com 1% dos coeficientes, no entanto, ocorre perda significativa de qualidade, com redução de detalhes e distorção perceptível tornando uma percepção de áudio "abafado" ao usuário ouvinte da música. Com isso, não é uma boa experiência para o mesmo caso seja feita essa modalidade de compressão.

Isso ocorre porque a redução do número de coeficientes implica perda de informação no domínio da frequência o que afeta significativamente a resolução do áudio comprimido.

4 Conclusão

Os resultados obtidos demonstram a eficiência da Transformada de Fourier em aplicações de processamento de sinais e imagens e áudio.

No caso da imagem, a filtragem no domínio da frequência permite controlar o nível de detalhamento da reconstrução. Já no áudio, a compressão baseada na seleção de coeficientes reduz de forma perceptível a quantidade de dados, mantendo a resolução do áudio aceitável dependendo da porcentagem utilizada.

Portanto, essas técnicas são amplamente aplicadas em áreas como compressão de dados, processamento de imagens e sistemas de comunicação o que permite aumentar o armazenamento e o desempenho de dispositivos.

A Código MATLAB desenvolvido

O código completo utilizado na resolução do exercício está apresentado abaixo.

```
1 %% EXERCICIO 5 - SEL0615
2 % B rbara Fernandes Madera - 11915032
3 %
4 % Este script resolve:
5 % 1) Processamento de imagem RGB
6 % 2) Compress o de udio mantendo 5% e 1% dos coeficientes
7 %
8 % Arquivo analisado na pasta:
9 % - imagem.jpg
10 % - musica.wav
11
12 clc;
13 clear;
14 close all;
15
16 %% =====
17 % PARTE 1 - IMAGEM RGB
18 % =====
19
20 % Leitura da imagem
21 img = imread('imagem.jpg');
22
23 % Garantia de que a imagem est em RGB
24 if size(img,3) ~= 3
25     error('A imagem precisa ser colorida no formato RGB.');
26 end
27
28 % Separa o dos canais
29 R = img(:,:,1);
30 G = img(:,:,2);
31 B = img(:,:,3);
32
33 %% 1(a) Mostrar os tr s canais de cor
34 figure;
35 subplot(2,2,1);
36 imshow(img);
37 title('Imagem RGB original');
38
39 subplot(2,2,2);
40 imshow(R);
41 title('Canal R');
42
43 subplot(2,2,3);
```

```

44 imshow(G);
45 title('Canal G');
46
47 subplot(2,2,4);
48 imshow(B);
49 title('Canal B');
50
51 %% 1(b) Criar duas mscaras com raios diferentes
52 % Aqui escolhi dois raios:
53 raio1 = 25;
54 raio2 = 45;
55
56 % Tamanho da mscara
57 % Usamos 2*raio + 1 para ter centro bem definido
58 tam1 = 2*raio1 + 1;
59 tam2 = 2*raio2 + 1;
60
61 % Cria o das mscaras circulares
62 mascara1 = criarMascaraCircular(tam1, raio1);
63 mascara2 = criarMascaraCircular(tam2, raio2);
64
65 % Mostrar as mscaras
66 figure;
67 subplot(1,2,1);
68 imshow(mascara1, []);
69 title(['Mascara circular - raio = ', num2str(raio1)]);
70
71 subplot(1,2,2);
72 imshow(mascara2, []);
73 title(['Mascara circular - raio = ', num2str(raio2)]);
74
75 %% 1(c) Aplicar as mscaras para cada canal e reconstruir a
       imagem RGB filtrada
76 % A aplica o ser feita no dominio da frequencia.
77 % Cada canal transformado por FFT, filtrado e reconstruido.
78
79 % Filtragem com a mscara 1
80 R_filtrado_1 = aplicarMascaraFrequencia(double(R), mascara1);
81 G_filtrado_1 = aplicarMascaraFrequencia(double(G), mascara1);
82 B_filtrado_1 = aplicarMascaraFrequencia(double(B), mascara1);
83
84 img_filtrada_1 = cat(3, ...
85     uint8(ajustarIntervalo(R_filtrado_1)), ...
86     uint8(ajustarIntervalo(G_filtrado_1)), ...
87     uint8(ajustarIntervalo(B_filtrado_1)));
88
89 % Filtragem com a mscara 2
90 R_filtrado_2 = aplicarMascaraFrequencia(double(R), mascara2);
91 G_filtrado_2 = aplicarMascaraFrequencia(double(G), mascara2);
92 B_filtrado_2 = aplicarMascaraFrequencia(double(B), mascara2);
93

```

```

94 img_filtrada_2 = cat(3, ...
95     uint8(ajustarIntervalo(R_filtrado_2)), ...
96     uint8(ajustarIntervalo(G_filtrado_2)), ...
97     uint8(ajustarIntervalo(B_filtrado_2)));
98
99 % Mostrar as imagens filtradas
100 figure;
101 subplot(1,3,1);
102 imshow(img);
103 title('Imagem original');
104
105 subplot(1,3,2);
106 imshow(img_filtrada_1);
107 title(['Imagem filtrada - raio = ', num2str(raio1)]);
108
109 subplot(1,3,3);
110 imshow(img_filtrada_2);
111 title(['Imagem filtrada - raio = ', num2str(raio2)]);
112
113 %% =====
114 % PARTE 2 - AUDIO
115 % =====
116
117 % Leitura do audio
118 [audio, fs] = audioread('musica.wav');
119
120 % Se o audio tiver dois canais, transformar em mono para
    simplificar
121 if size(audio,2) == 2
122     audio = mean(audio, 2);
123 end
124
125 N = length(audio);
126
127 %% FFT do audio
128 X = fft(audio);
129
130 % Magnitude dos coeficientes
131 magX = abs(X);
132
133 %% 2(a) Compress o mantendo 5% dos coeficientes de maior
    magnitude
134 percentual1 = 0.05;
135 audio_5 = comprimirAudioFFT(audio, percentual1);
136
137 % Salvar
138 audiowrite('musica_comprimida_5porcento.wav', audio_5, fs);
139
140 %% 2(b) Compress o mantendo 1% dos coeficientes de maior
    magnitude
141 percentual2 = 0.01;

```

```

142 audio_1 = comprimirAudioFFT(audio, percentual2);
143
144 % Salvar
145 audiowrite('musica_comprimida_1porcento.wav', audio_1, fs);
146
147 %% Mostrar compara o no tempo
148 t = (0:N-1)/fs;
149
150 figure;
151 subplot(3,1,1);
152 plot(t, audio);
153 title('Audio original');
154 xlabel('Tempo (s)');
155 ylabel('Amplitude');
156
157 subplot(3,1,2);
158 plot(t, audio_5);
159 title('Audio comprimido - 5% dos coeficientes');
160 xlabel('Tempo (s)');
161 ylabel('Amplitude');
162
163 subplot(3,1,3);
164 plot(t, audio_1);
165 title('Audio comprimido - 1% dos coeficientes');
166 xlabel('Tempo (s)');
167 ylabel('Amplitude');
168
169 %% Mostrar espectros
170 X_5 = fft(audio_5);
171 X_1 = fft(audio_1);
172 f = (0:N-1)*(fs/N);
173
174 figure;
175 subplot(3,1,1);
176 plot(f(1:floor(N/2)), abs(X(1:floor(N/2))));
177 title('Espectro do audio original');
178 xlabel('Frequencia (Hz)');
179 ylabel('Magnitude');
180
181 subplot(3,1,2);
182 plot(f(1:floor(N/2)), abs(X_5(1:floor(N/2))));
183 title('Espectro do audio comprimido - 5%');
184 xlabel('Frequencia (Hz)');
185 ylabel('Magnitude');
186
187 subplot(3,1,3);
188 plot(f(1:floor(N/2)), abs(X_1(1:floor(N/2))));
189 title('Espectro do audio comprimido - 1%');
190 xlabel('Frequencia (Hz)');
191 ylabel('Magnitude');
192

```



```

193 %% Informacoes finais
194 fprintf('Arquivos gerados com sucesso:\n');
195 fprintf('- musica_comprimida_5porcento.wav\n');
196 fprintf('- musica_comprimida_1porcento.wav\n');
197
198 %% =====
199 % FUNCOES LOCAIS
200 % =====
201
202 function mascara = criarMascaraCircular(tamanho, raio)
203 % Cria uma mascara circular binaria
204
205     centro = ceil(tamanho/2);
206     mascara = zeros(tamanho, tamanho);
207
208     for i = 1:tamanho
209         for j = 1:tamanho
210             dist = sqrt((i-centro)^2 + (j-centro)^2);
211             if dist <= raio
212                 mascara(i,j) = 1;
213             end
214         end
215     end
216 end
217
218 function img_filtrada = aplicarMascaraFrequencia(img,
219     mascaraPequena)
220 % Aplica uma mascara no dominio da frequencia
221 % A mascara pequena centralizada e expandida para o tamanho
222 % da imagem
223
224     [M, N] = size(img);
225
226     % FFT e centraliza o
227     F = fft2(img);
228     Fshift = fftshift(F);
229
230     % Criar mascara do tamanho da imagem
231     mascara = zeros(M, N);
232
233     [mMask, nMask] = size(mascaraPequena);
234
235     iniM = floor((M - mMask)/2) + 1;
236     fimM = iniM + mMask - 1;
237     iniN = floor((N - nMask)/2) + 1;
238     fimN = iniN + nMask - 1;
239
240     mascara(iniM:fimM, iniN:fimN) = mascaraPequena;
241
242     % Aplicar filtro
243     Ffiltrado = Fshift .* mascara;

```

```

242
243     % Volta para dom nio da imagem
244     img_filtrada = real(ifft2(ifftshift(Ffiltrado)));
245 end
246
247 function saida = ajustarIntervalo(img)
248 % Ajusta a imagem para o intervalo [0,255]
249
250     img = img - min(img(:));
251     if max(img(:)) > 0
252         img = img / max(img(:));
253     end
254     saida = 255 * img;
255 end
256
257 function audio_comprimido = comprimirAudioFFT(audio, percentual)
258 % Mant m apenas uma porcentagem dos maiores coeficientes em
    magnitude
259
260     X = fft(audio);
261     N = length(X);
262
263     % N mero de coeficientes a manter
264     K = round(percentual * N);
265
266     % Ordenar magnitudes
267     [~, indices] = sort(abs(X), 'descend');
268
269     % Criar vetor zerado
270     X_comp = zeros(size(X));
271
272     % Manter apenas os K maiores coeficientes
273     X_comp(indices(1:K)) = X(indices(1:K));
274
275     % Reconstru o
276     audio_comprimido = real(ifft(X_comp));
277
278     % Normalizar para evitar clipping
279     maxVal = max(abs(audio_comprimido));
280     if maxVal > 0
281         audio_comprimido = audio_comprimido / maxVal;
282     end
283 end

```